

UNIVERSIDAD ESTATAL AMAZÓNICA

Unidad de Organización Curricular: Básica

Campo de Estudio: Básica

INFORME DE PRÁCTICAS DEL COMPONENTE PRÁCTICO-EXPERIMENTAL	
Nombre del Estudiante: ALEJANDRO BENJAMÍN ROBALINO ZALDUMBIDE	
Asignatura: Estructura de Datos (UEA-L-UFB-032)	Calificación:
Fecha del informe: 20 de julio de 2026	
Nº Práctica: 02	
Título de la Práctica: Implementación de Pilas - Botón retroceder/adelantar	

Introducción	El diseño de elementos de software requiere estructuras de datos eficientes para el almacenamiento temporal y la manipulación secuencial de la información. En el desarrollo con C# y la plataforma .NET, la clase genérica <i>Stack<T></i> proporciona una implementación de la estructura Pila, basada en el principio LIFO (Last-In, First-Out). Esta estructura garantiza que el último elemento agregado sea el primero en ser extraído, ideal para modelar flujos de navegación secuencial reversible. En esta práctica, se propone una mejora sustancial a la simulación del botón “retroceder” tradicional. Un navegador real no solo permite volver atrás en el historial, sino también “adelantar” (avanzar a las páginas que fueron retrocedidas). Para modelar este comportamiento bidireccional de manera eficiente, se implementa una arquitectura de doble pila: una pila de retroceso (<i>_historialAtras</i>) y una pila de avance (<i>_historialAdelante</i>). Este enfoque permite mantener un flujo dinámico y coherente de navegación con una complejidad temporal óptima de $O(1)$ en todas sus operaciones críticas.
--------------	--

Desarrollo o procedimiento	El sistema se desarrolló en lenguaje C# utilizando Programación Orientada a Objetos (POO) bajo la plataforma .NET 8. El entorno del sistema es una aplicación de escritorio de tipo consola interactiva que permite manipular el flujo del navegador en tiempo real. 1. Clase PaginaWeb: Representa el objeto de datos individual. Contiene las propiedades auto-implementadas de solo lectura <i>Url</i> (dirección web), <i>Titulo</i> (nombre de la página) y <i>FechaAcceso</i> (marca temporal automatizada). 2. Clase Navegador: Contiene la lógica del negocio. Utiliza dos pilas nativas de tipo <i>Stack<PaginaWeb></i> para manejar la navegación bidireccional. La propiedad <i>PaginaActual</i> mantiene el estado activo del navegador. 3. Operaciones de Navegación: ▪ VisitarPagina: Al ingresar a un nuevo sitio, la página actual se apila en <i>_historialAtras</i>, la nueva página se establece como actual y se limpia por completo la pila <i>_historialAdelante</i> (bifurcación del historial). ▪ Retroceder: Si la pila de atrás no está vacía, la página actual se apila en <i>_historialAdelante</i> y se realiza un <i>Pop</i> de <i>_historialAtras</i> para ser asignada a <i>PaginaActual</i>. ▪ Adelantar: Si la pila de adelante tiene elementos, la página actual se apila en <i>_historialAtras</i> y se realiza un <i>Pop</i> de <i>_historialAdelante</i> para actualizar la <i>PaginaActual</i>. 4. Reportería y Benchmarking: Se implementó el método <i>MostrarHistorial</i> que realiza una lectura no destructiva sobre las pilas mediante iteradores de C#. Además, se desarrolló una prueba de rendimiento en tiempo real utilizando la clase <i>Stopwatch</i> para medir los ticks de CPU y nanosegundos en un escenario de 100,000 transiciones de navegación. Agente de IA utilizado: Se utilizó el asistente de IA Antigravity de Google DeepMind para optimizar la estructura del proyecto en C# (.NET 8 con tipado nullable habilitado para prevenir excepciones por referencia nula) y estructurar el presente informe en formato LaTeX. Se estima una asistencia del 35 % en la estructuración de la lógica y 100 % en la maquetación del documento.
Resultados	La simulación interactiva permitió comprobar la coherencia lógica de las dos pilas. Al alternar entre visitas, retrocesos y avances, los estados del navegador se actualizaron sin pérdida de datos. • Reportería en tiempo real: El menú de consola visualizó el contenido exacto de ambas pilas de forma transparente. • Tiempos de ejecución: Las mediciones del Stopwatch indicaron que tanto la inserción como la extracción individuales toman menos de 1 microsegundo.

	• Prueba de escala (Benchmark): La prueba de 100,000 inserciones consecutivas arrojó tiempos promedio de 15 a 45 nanosegundos por elemento. La extracción consecutiva mantuvo un promedio similar, demostrando empíricamente que la complejidad algorítmica se mantiene en $O(1)$ constante, sin importar la cantidad de elementos en memoria.
Conclusiones	La estructura de datos Pila es la más adecuada para simular el historial secuencial inverso de un navegador, y la adición de una segunda pila completa el ciclo de retroceso/avance de manera óptima. • Ventajas: Las operaciones <i>Push</i> y <i>Pop</i> en la estructura Stack tienen una complejidad de $O(1)$, lo que garantiza respuestas inmediatas en tiempo constante, ideales para sistemas operativos y navegadores de alta carga de datos. • Desventajas: Las pilas no permiten el acceso aleatorio. Para buscar o acceder a un elemento en medio de la pila, es necesario vaciar temporalmente los elementos superiores, lo que destruiría el historial si no se implementaran estructuras complementarias de almacenamiento.
Referencias Bibliográficas	Castillo Barrios, J. I. (2022). <i>Método adaptivo-reactivo de replicación para sistemas de almacenamiento de alta disponibilidad</i> [Tesis de maestría, Centro de Investigación y de Estudios Avanzados del Instituto Politécnico Nacional]. Repositorio Institucional. García, B. C. (2025). <i>Diseño de elementos software con tecnologías basadas en componentes</i>. UF1289. TUTOR FORMACIÓN. Jamro, M. (2024). <i>C# Data Structures and Algorithms: Harness the power of C# to build a diverse range of efficient applications</i>. Packt Publishing Ltd. Jiménez Suárez, M. A. (2025). <i>Análisis de algoritmos para la generación de trayectorias SAR con IA</i> [Trabajo de fin de grado]. Repositorio Institucional.
Anexos	Repositorio de Código: https://github.com/Rodrigo2461/PE-ESTRUCTURA-DE-DATOS. git

Anexo: Código Fuente en C# (Program.cs)

```
1 using System;
2 using System.Collections.Generic;
3 using System.Diagnostics;
4
5 namespace SimulacionNavegador
6 {
7     /// <summary>
8     /// Representa un sitio web visitado en el simulador de navegación.
9     /// </summary>
10    public class PaginaWeb
11    {
12        public string Url { get; }
13        public string Titulo { get; }
14        public DateTime FechaAcceso { get; }
15
16        public PaginaWeb(string url, string titulo)
17        {
18            Url = string.IsNullOrEmpty(url) ? "about:blank" : url;
19            Titulo = string.IsNullOrEmpty(titulo) ? "Pestaña Nueva" : titulo;
20            FechaAcceso = DateTime.Now;
21        }
22
23        public override string ToString()
24        {
25            return $"{Titulo} ({Url}) - Accedido: {FechaAcceso:HH:mm:ss}";
26        }
27    }
28
29    /// <summary>
30    /// Administra la navegación utilizando dos estructuras tipo Pila (Stack)
31    /// para permitir la navegación hacia atrás y hacia adelante con complejidad O(1)
32    .
33    /// </summary>
34    public class Navegador
35    {
36        private readonly Stack<PaginaWeb> _historialAtras;
37        private readonly Stack<PaginaWeb> _historialAdelante;
38
39        public PaginaWeb? PaginaActual { get; private set; }
40
41        public int CantidadAtras => _historialAtras.Count;
42        public int CantidadAdelante => _historialAdelante.Count;
43
44        public Navegador()
45        {
46            _historialAtras = new Stack<PaginaWeb>();
47            _historialAdelante = new Stack<PaginaWeb>();
48            PaginaActual = null;
49        }
50
51        /// <summary>
52        /// Visita una nueva página. Limpia el historial de avance (Adelante)
53        /// ya que se inicia una nueva bifurcación en el camino de navegación.
54        /// </summary>
55        /// <param name="nuevaPagina">La página web a visitar.</param>
56        /// <returns>Los ticks de CPU que tomó la operación.</returns>
57        public long VisitarPagina(PaginaWeb nuevaPagina)
58        {
59            Stopwatch cronometro = Stopwatch.StartNew();
60
61            if (PaginaActual != null)
62            {
63                _historialAtras.Push(PaginaActual);
```

```

64         PaginaActual = nuevaPagina;
65         _historialAdelante.Clear(); // Rompe la linealidad de avance anterior
66
67         cronometro.Stop();
68         return cronometro.ElapsedTicks;
69     }
70
71     /// <summary>
72     /// Retrocede a la página anterior si existe.
73     /// </summary>
74     /// <returns>Una tupla indicando éxito, la página cargada (si aplica) y los
75     /// ticks transcurridos.</returns>
76     public (bool exito, PaginaWeb? paginaCargada, long ticks) Retroceder()
77     {
78         Stopwatch cronometro = Stopwatch.StartNew();
79
80         if (_historialAtras.Count > 0)
81         {
82             if (PaginaActual != null)
83             {
84                 _historialAdelante.Push(PaginaActual);
85             }
86             PaginaActual = _historialAtras.Pop();
87             cronometro.Stop();
88             return (true, PaginaActual, cronometro.ElapsedTicks);
89         }
90
91         cronometro.Stop();
92         return (false, null, cronometro.ElapsedTicks);
93     }
94
95     /// <summary>
96     /// Avanza a la siguiente página en el historial de adelante si existe.
97     /// </summary>
98     /// <returns>Una tupla indicando éxito, la página cargada (si aplica) y los
99     /// ticks transcurridos.</returns>
100     public (bool exito, PaginaWeb? paginaCargada, long ticks) Adelantar()
101     {
102         Stopwatch cronometro = Stopwatch.StartNew();
103
104         if (_historialAdelante.Count > 0)
105         {
106             if (PaginaActual != null)
107             {
108                 _historialAtras.Push(PaginaActual);
109             }
110             PaginaActual = _historialAdelante.Pop();
111             cronometro.Stop();
112             return (true, PaginaActual, cronometro.ElapsedTicks);
113         }
114
115         cronometro.Stop();
116         return (false, null, cronometro.ElapsedTicks);
117     }
118
119     /// <summary>
120     /// Limpia todo el historial de navegación y la página actual.
121     /// </summary>
122     public void LimpiarHistorial()
123     {
124         _historialAtras.Clear();
125         _historialAdelante.Clear();
126         PaginaActual = null;
127     }
128
129     /// <summary>

```

```

128     /// Retorna el historial de retroceso.
129     /// </summary>
130     public IEnumerable<PaginaWeb> ObtenerHistorialAtras()
131     {
132         return _historialAtras;
133     }
134
135     /// <summary>
136     /// Retorna el historial de avance.
137     /// </summary>
138     public IEnumerable<PaginaWeb> ObtenerHistorialAdelante()
139     {
140         return _historialAdelante;
141     }
142 }
143
144 class Program
145 {
146     static void Main(string[] args)
147     {
148         Navegador navegador = new Navegador();
149         bool salir = false;
150
151         // Inicializar con algunos datos de ejemplo
152         navegador.VisitarPagina(new PaginaWeb("https://www.google.com", "Google"));
153         navegador.VisitarPagina(new PaginaWeb("https://www.github.com", "GitHub"));
154         navegador.VisitarPagina(new PaginaWeb("https://stackoverflow.com", "Stack
            Overflow"));
155
156         while (!salir)
157         {
158             Console.Clear();
159             Console.ForegroundColor = ConsoleColor.Cyan;
160             Console.WriteLine("
                =====
            ");
161             Console.WriteLine("          SIMULADOR DE BOTÓN RETROCEDER Y ADELANTAR
                DE NAVEGADOR          ");
162             Console.WriteLine("
                =====
            ");
163             Console.ResetColor();
164
165             Console.Write("\nPÁGINA ACTUAL: ");
166             if (navegador.PaginaActual != null)
167             {
168                 Console.ForegroundColor = ConsoleColor.Green;
169                 Console.WriteLine(navegador.PaginaActual);
170             }
171             else
172             {
173                 Console.ForegroundColor = ConsoleColor.DarkGray;
174                 Console.WriteLine("[Ninguna página cargada (about:blank)]");
175             }
176             Console.ResetColor();
177
178             Console.WriteLine("\n--- REPORTERÍA DEL ESTADO DE LAS PILAS ---");
179
180             Console.ForegroundColor = ConsoleColor.DarkYellow;
181             Console.WriteLine($"[PILA ADELANTE] (Elementos: {navegador.
                CantidadAdelante})");
182             if (navegador.CantidadAdelante == 0)
183             {
184                 Console.WriteLine("    <- (Vacío - No se puede adelantar)");

```

```

185     }
186     else
187     {
188         int i = 1;
189         foreach (var pag in navegador.ObtenerHistorialAdelante())
190         {
191             Console.WriteLine($" [{i}] {pag.Titulo} ({pag.Url})");
192             i++;
193         }
194     }
195
196     Console.WriteLine();
197
198     Console.ForegroundColor = ConsoleColor.Magenta;
199     Console.WriteLine($"[PILA ATRÁS] (Elementos: {navegador.CantidadAtras
200     })");
201     if (navegador.CantidadAtras == 0)
202     {
203         Console.WriteLine(" <- (Vacío - No se puede retroceder)");
204     }
205     else
206     {
207         int i = 1;
208         foreach (var pag in navegador.ObtenerHistorialAtras())
209         {
210             Console.WriteLine($" [{i}] {pag.Titulo} ({pag.Url})");
211             i++;
212         }
213     }
214     Console.ResetColor();
215     Console.WriteLine("-----");
216
217     Console.WriteLine("\nMENÚ DE OPCIONES:");
218     Console.WriteLine("1. Visitar una nueva página web");
219     Console.WriteLine("2. Retroceder (Atrás)");
220     Console.WriteLine("3. Adelantar (Adelante)");
221     Console.WriteLine("4. Limpiar historial de navegación");
222     Console.WriteLine("5. Ejecutar simulación de rendimiento (
223     Benchmarking)");
224     Console.WriteLine("6. Salir");
225     Console.Write("\nSeleccione una opción: ");
226
227     string opcion = Console.ReadLine() ?? "";
228     switch (opcion)
229     {
230     case "1":
231         Console.Write("\nIngrese la URL del sitio (ej. https://
232         example.com): ");
233         string url = Console.ReadLine() ?? "";
234         Console.Write("Ingrese el título del sitio (ej. Ejemplo): ");
235         string titulo = Console.ReadLine() ?? "";
236
237         if (string.IsNullOrEmpty(url))
238         {
239             Console.ForegroundColor = ConsoleColor.Red;
240             Console.WriteLine("\n[Error] La URL no puede estar vacía.
241             ");
242             Console.ResetColor();
243             PresioneTecla();
244             break;
245         }
246
247         long ticksVisita = navegador.VisitarPagina(new PaginaWeb(url,
248         titulo));
249         Console.ForegroundColor = ConsoleColor.Green;
250         Console.WriteLine($"[Éxito] Página visitada con éxito.");

```

```

246 Console.ForegroundColor = ConsoleColor.DarkGray;
247 Console.WriteLine($"Tiempo de procesamiento de inserción: {
    ticksVisita} ticks.");
248 Console.ResetColor();
249 PresioneTecla();
250 break;
251
252 case "2":
253     var resRetroceder = navegador.Retroceder();
254     if (resRetroceder.exito && resRetroceder.paginaCargada !=
        null)
255     {
256         Console.ForegroundColor = ConsoleColor.Green;
257         Console.WriteLine($"[Éxito] Retrocediendo a: {
            resRetroceder.paginaCargada.Titulo}");
258         Console.ForegroundColor = ConsoleColor.DarkGray;
259         Console.WriteLine($"Tiempo de retroceso: {resRetroceder.
            ticks} ticks.");
260     }
261     else
262     {
263         Console.ForegroundColor = ConsoleColor.Yellow;
264         Console.WriteLine($"[Alerta] No hay páginas en el
            historial para retroceder.");
265     }
266     Console.ResetColor();
267     PresioneTecla();
268     break;
269
270 case "3":
271     var resAdelantar = navegador.Adelantar();
272     if (resAdelantar.exito && resAdelantar.paginaCargada != null)
273     {
274         Console.ForegroundColor = ConsoleColor.Green;
275         Console.WriteLine($"[Éxito] Adelantando a: {
            resAdelantar.paginaCargada.Titulo}");
276         Console.ForegroundColor = ConsoleColor.DarkGray;
277         Console.WriteLine($"Tiempo de avance: {resAdelantar.ticks
            } ticks.");
278     }
279     else
280     {
281         Console.ForegroundColor = ConsoleColor.Yellow;
282         Console.WriteLine($"[Alerta] No hay páginas en el
            historial para adelantar.");
283     }
284     Console.ResetColor();
285     PresioneTecla();
286     break;
287
288 case "4":
289     navegador.LimpiarHistorial();
290     Console.ForegroundColor = ConsoleColor.Green;
291     Console.WriteLine($"[Éxito] El historial se ha limpiado por
        completo.");
292     Console.ResetColor();
293     PresioneTecla();
294     break;
295
296 case "5":
297     EjecutarBenchmark();
298     PresioneTecla();
299     break;
300
301 case "6":
302     salir = true;

```



```

303         Console.WriteLine("\nGracias por usar el simulador.");
304         break;
305
306         default:
307             Console.ForegroundColor = ConsoleColor.Red;
308             Console.WriteLine("\nOpción no válida. Intente de nuevo.");
309             Console.ResetColor();
310             PresioneTecla();
311             break;
312     }
313 }
314 }
315
316 static void PresioneTecla()
317 {
318     Console.WriteLine("\nPresione cualquier tecla para continuar...");
319     Console.ReadKey();
320 }
321
322 static void EjecutarBenchmark()
323 {
324     Console.Clear();
325     Console.ForegroundColor = ConsoleColor.Cyan;
326     Console.WriteLine("
=====
");
327     Console.WriteLine("                SIMULACIÓN DE RENDIMIENTO (BENCHMARK DE LA
PILA)
");
328     Console.WriteLine("
=====
");
329     Console.ResetColor();
330     Console.WriteLine("Esta simulación realiza inserciones (Push) y
extracciones (Pop) a gran");
331     Console.WriteLine("escala para demostrar empíricamente la complejidad O
(1) de las pilas.");
332     Console.WriteLine("Se insertarán y extraerán 100,000 páginas en el
historial.\n");
333
334     Navegador navTest = new Navegador();
335     int numElementos = 100000;
336
337     Stopwatch swPush = Stopwatch.StartNew();
338     for (int i = 0; i < numElementos; i++)
339     {
340         navTest.VisitarPagina(new PaginaWeb($"https://site{i}.com", $"Sitio {
i}"));
341     }
342     swPush.Stop();
343
344     double promedioPushNs = (double)swPush.ElapsedTicks / numElementos *
(1000000000.0 / Stopwatch.Frequency);
345     Console.WriteLine($"- Inserción (Push) de {numElementos:N0} elementos:");
346     Console.WriteLine($" * Tiempo total: {swPush.ElapsedMilliseconds} ms ({
swPush.ElapsedTicks:N0} ticks)");
347     Console.WriteLine($" * Tiempo promedio por Push: {promedioPushNs:F2} ns"
);
348     Console.WriteLine($" * Complejidad experimental: O(1) [Constante]");
349
350     Console.WriteLine();
351
352     Stopwatch swPop = Stopwatch.StartNew();
353     for (int i = 0; i < numElementos - 1; i++)
354     {
355         navTest.Retroceder();
356     }

```

```

357         swPop.Stop();
358
359         double promedioPopNs = (double)swPop.ElapsedTicks / (numElementos - 1) *
360             (1000000000.0 / Stopwatch.Frequency);
361         Console.WriteLine($"- Extracción (Pop) de {numElementos - 1:N0} elementos
362             :");
363         Console.WriteLine($" * Tiempo total: {swPop.ElapsedMilliseconds} ms ({
364             swPop.ElapsedTicks:N0} ticks)");
365         Console.WriteLine($" * Tiempo promedio por Pop: {promedioPopNs:F2} ns");
366         Console.WriteLine($" * Complejidad experimental: O(1) [Constante]");
367
368         Console.ForegroundColor = ConsoleColor.Green;
369         Console.WriteLine("\n[Conclusión] Las operaciones no dependen del número
370             de elementos almacenados,");
371         Console.WriteLine("lo que valida teórica y experimentalmente que Stack
372             posee un rendimiento O(1).");
373         Console.ResetColor();
374     }
375 }

```